

Structuri de Date

Anul universitar 2019-2020

Prof. Adina Magda Florea



Curs Nr. 11

Tabele de dispersie

- ☐ Căutări după cheie
- ☐ Tabele de dispersie
- ☐ Operații specifice
- ☐ Funcții de dispersie
- ☐ Căutări în șiruri
 - ☐ Algoritmul Rabin-Karp
 - ☐ Algoritmul LZV
 - ☐ Algoritmul Knuth-Morris-Pratt

1. Căutari dupa cheie

Multimi de elemente in care se efectueaza cautari frecvente cu "cheie" sir de caractere:

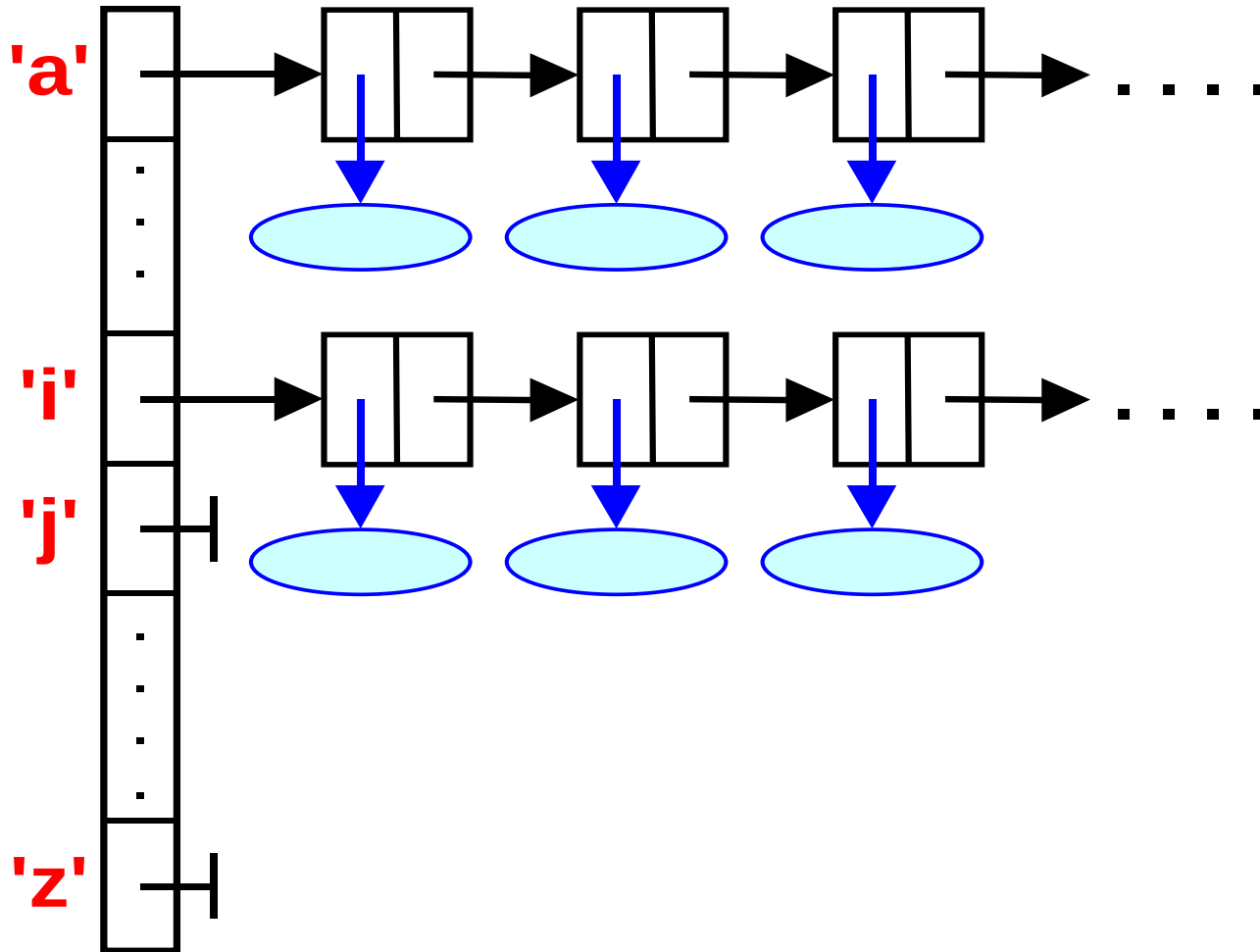
- dictionar
- agenda telefonica
- index de strazi / localitati

Pentru flexibilitate si eficienta operatiilor de inserare si eliminare se poate utiliza o lista.

Pentru eficienta operatiilor de cautare nu este indicata utilizarea unei singure liste.

Solutia deja discutata: utilizarea unui **vector de liste**, in care elementele sunt grupate dupa initiala cheii.

Chei grupate dupa inițială



2. Tabele de dispersie

Problema: **durata cautarii** este dependenta de lungimea listei in care se cauta, iar in majoritatea cazurilor lungimile listelor sunt foarte diferite – unele liste pot fi foarte lungi, in timp ce altele pot fi vide.

Solutie: utilizarea unei functii de **dispersie** (*hash function*).

Funcția de dispersie generează, pentru orice **cheie** (sir de caractere), un **cod de dispersie** dintr-un domeniu fixat **[0,M-1]**.

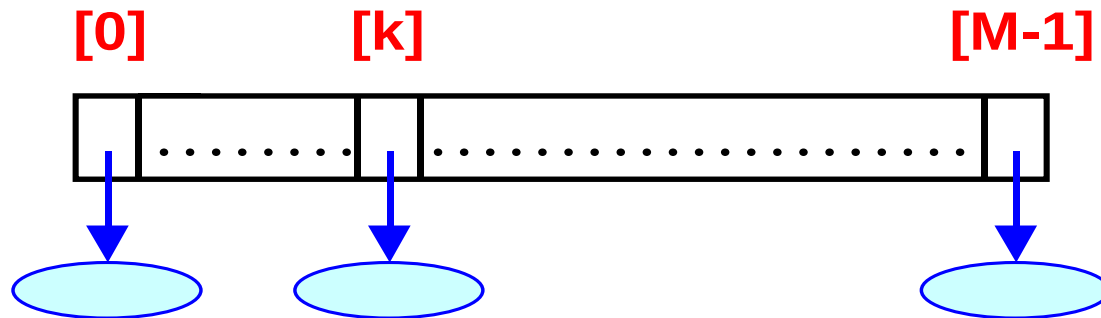
Tabele de dispersie

Caz particular

M este chiar **numarul total** de elemente.

Funcția de dispersie genereaza un **cod distinct** pentru fiecare element.

Intr-un astfel de caz elementele pot fi memorate direct intr-un **vector** numit **tabela de dispersie cu adresare directa**, **indicele** unui element fiind reprezentat de codul de dispersie corespunzator.

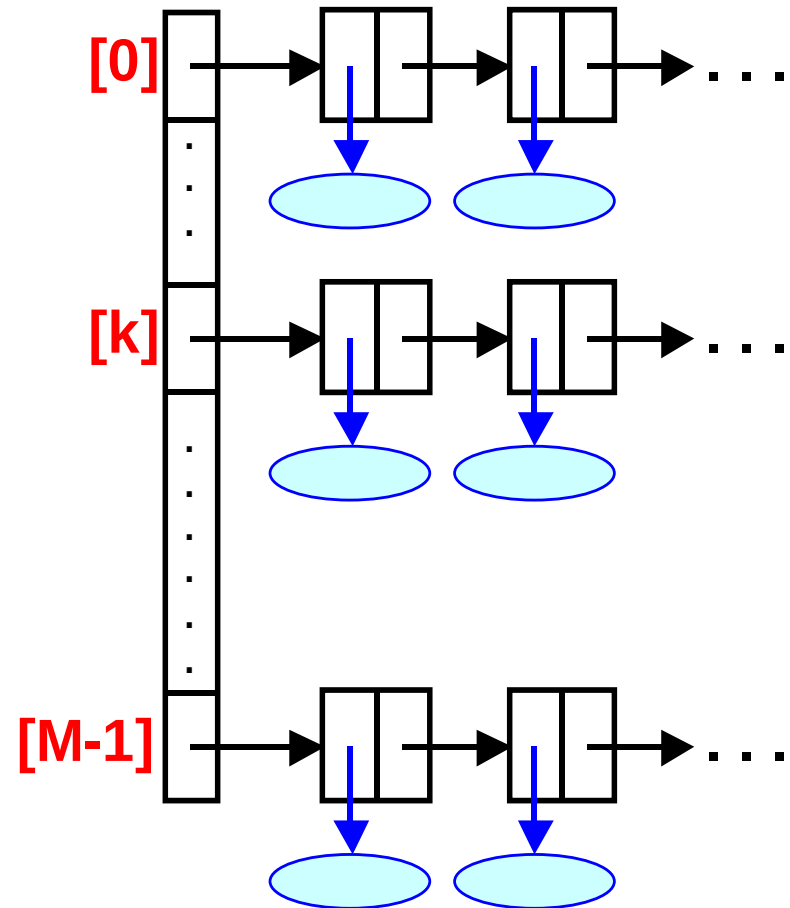


Tabele de dispersie

În cazul general **numarul de valori distincte** ale codului de dispersie este **mai mic** decat **numarul total de elemente**.

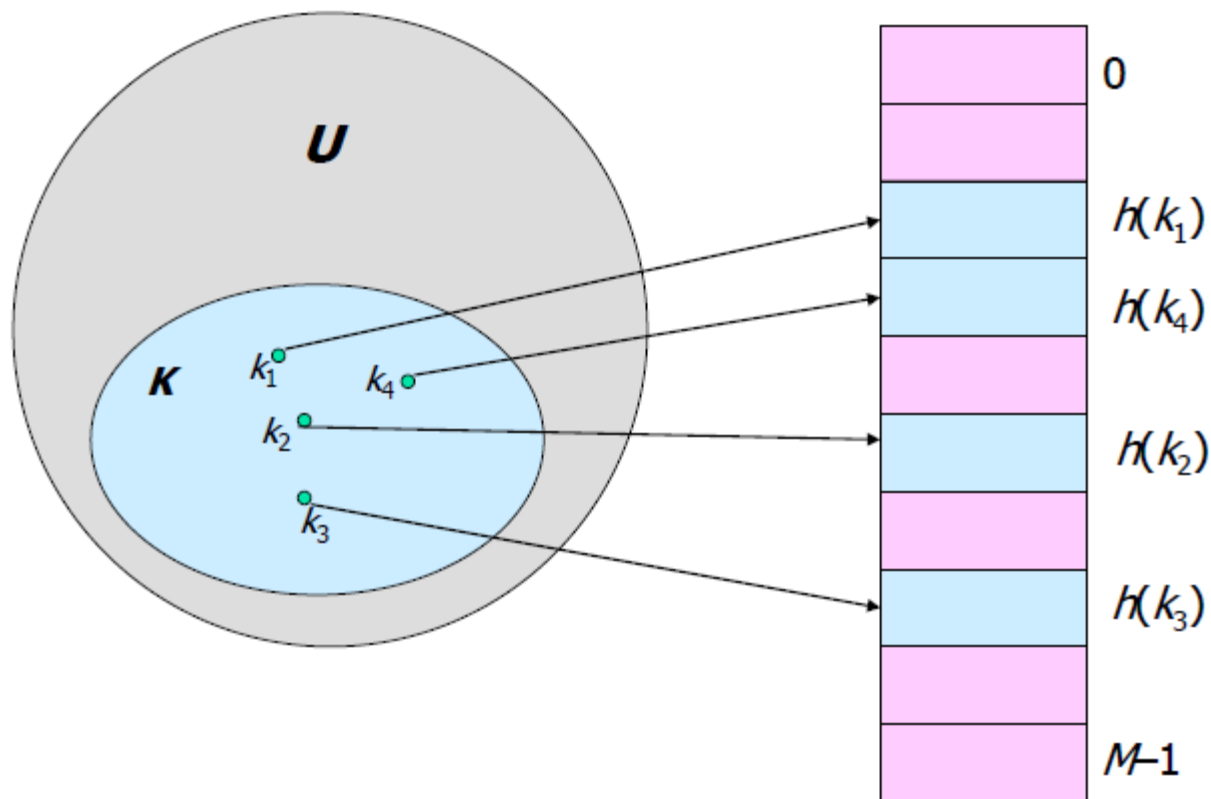
În acest caz între două elemente cu același cod de dispersie **k** se produce o **coliziune**.

Cea mai simplă metodă de rezolvare a coliziunilor constă în **adresare prin înlanțuire**.

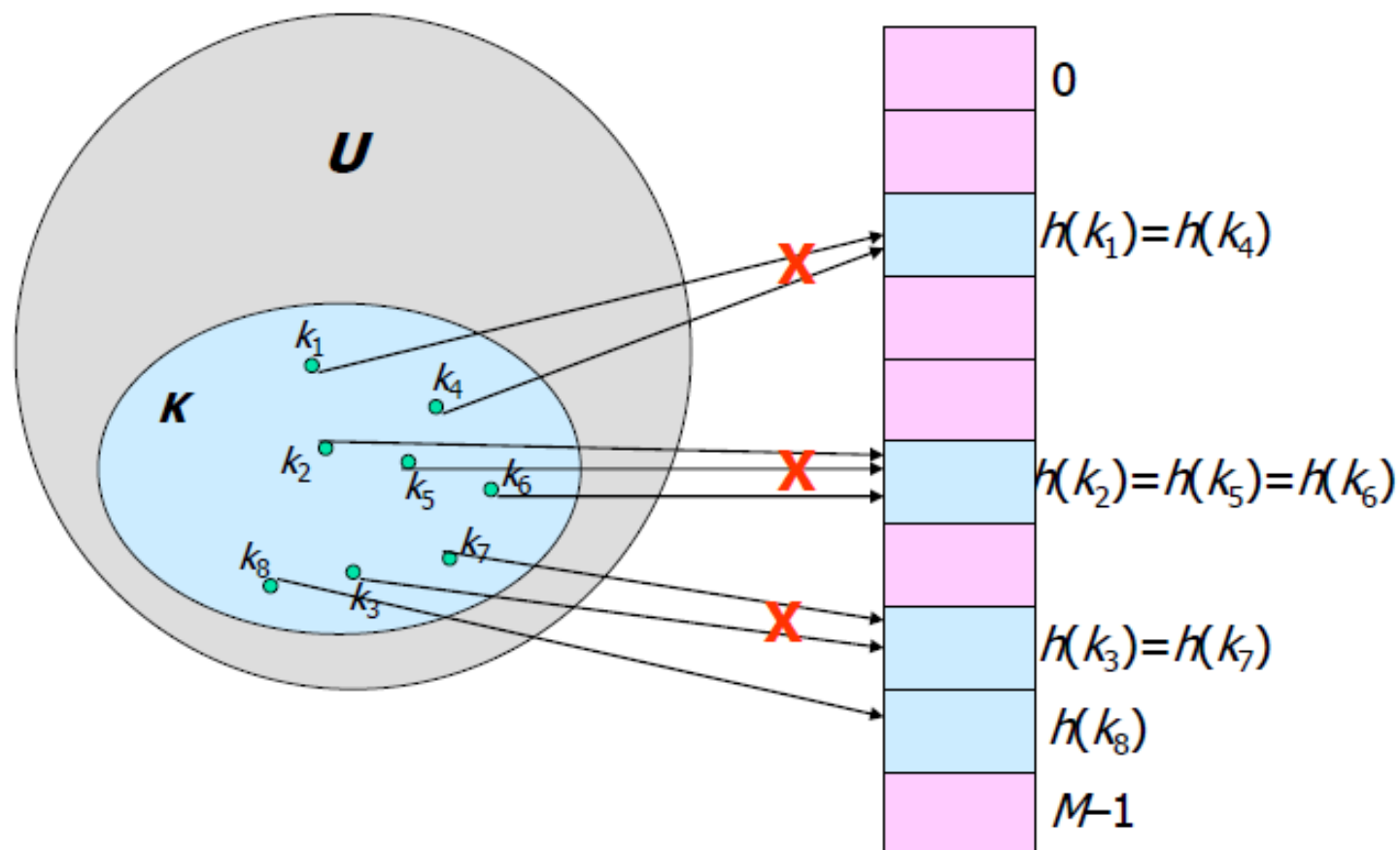


U – multimea de chei

K – codul de dispersie (h - functia care genereaza codul)

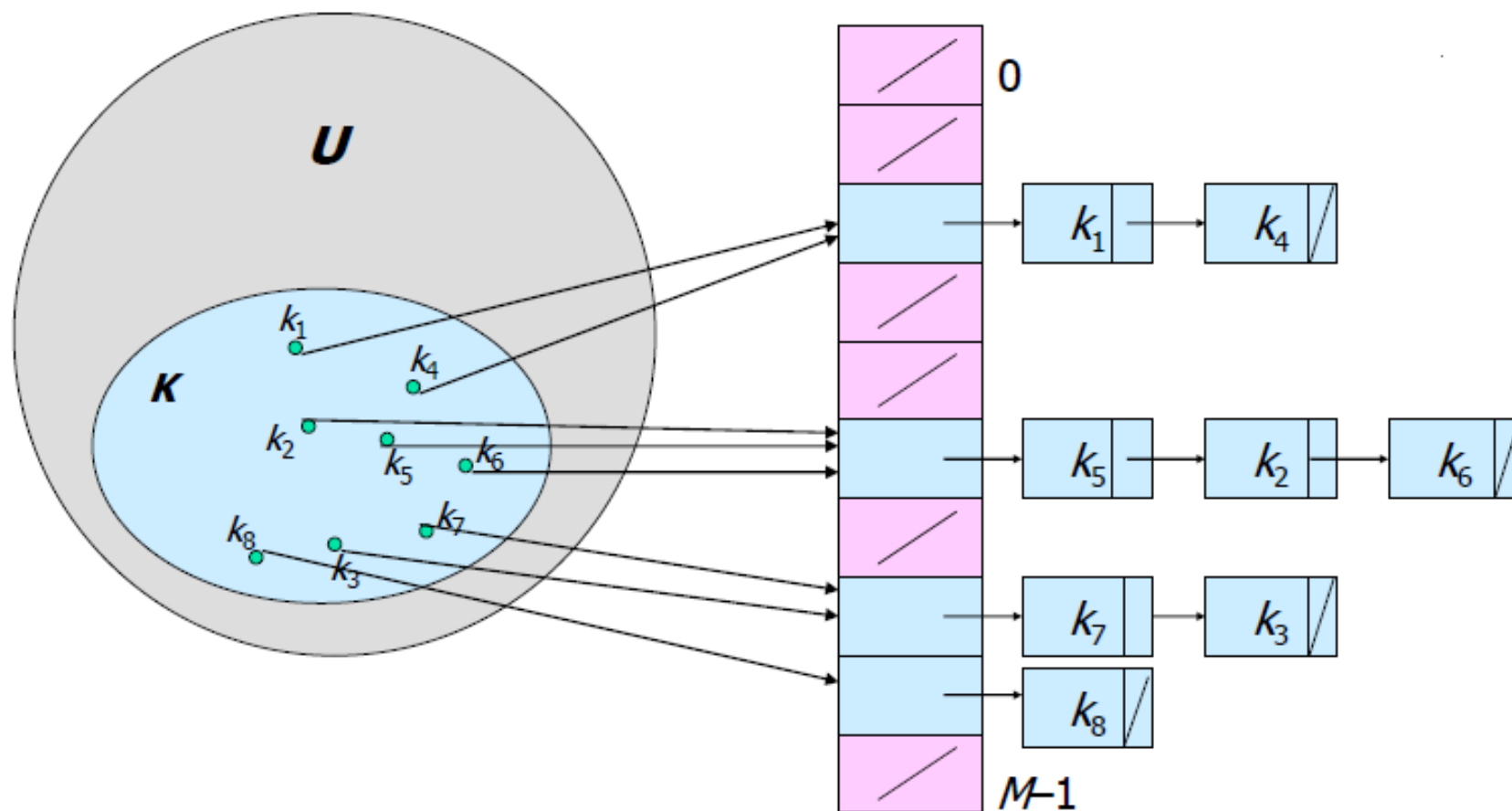


Colizione



Rezolvarea coliziunilor

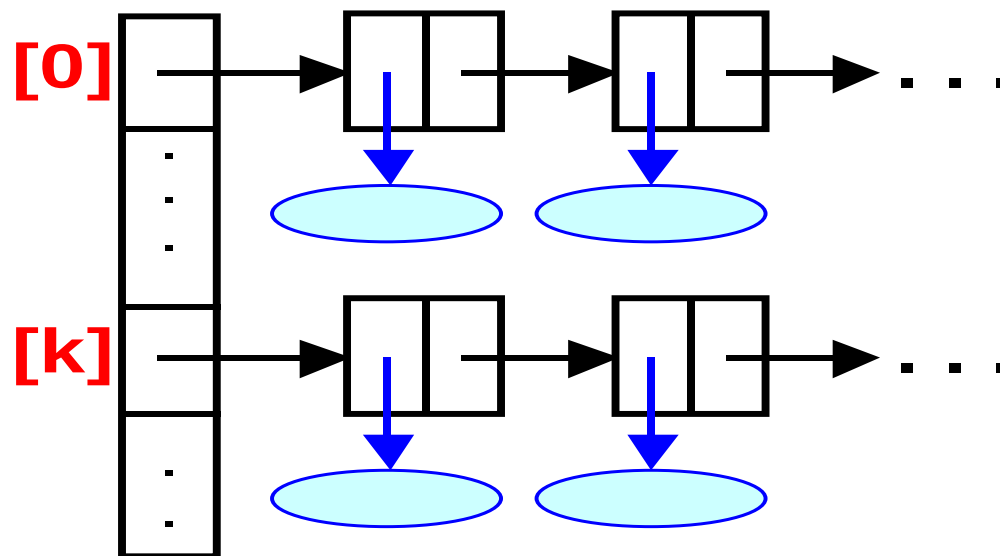
Adresare prin inlantuire



Tabele de dispersie

Elementele tablei de dispersie sunt adrese de liste, in fiecare lista fiind memorate adresele elementelor cu **acelasi cod de dispersie** $k \in [0, M-1]$.

Pentru eficienta cautarii oricaruia dintre cele **N** elemente este de dorit ca **lungimile celor M liste** sa fie **cat mai apropiate** de valoarea $N/M = \text{factor de incarcare}$ (*load factor*) al tablei de dispersie.



Reprezentare

```
typedef char* Key;  
typedef int Item;  
typedef long (*HashFunction)(char*, long);
```

- Intrarea intr-o tabela de dispersie

```
typedef struct cel {  
    Key key;  
    Item elem;  
    struct cel *next; } T Cel, *AT Cel;
```

- Tabela de dispersie

```
typedef struct HashTable {  
    AT Cel* elements;  
    long size;  
    HashFunction hashFunction; } HashTable;
```

3. Tabele de dispersie – operații specifice

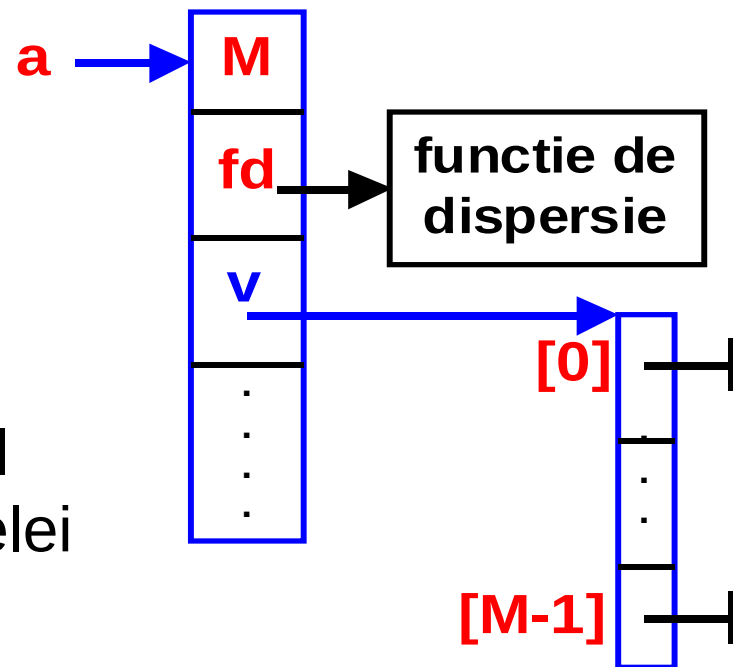
Initializare:

➤ parametri:

- dimensiunea tablei (**M**)
- functia de dispersie (**fd**)
- alte elemente, specifice implementarii

➤ efect: alocă și initializează spațiul de memorie pentru descriptorul tablei și vectorul adreselor de liste

➤ rezultat: adresa descriptor sau NULL



Tabele de dispersie – operații specifice

Inserare:

➤ parametri:

- adresa tabela
- element

➤ **efect:** calculeaza codul de dispersie si actualizeaza lista corespunzatoare, daca elementul nu exista deja

➤ rezultat:

- 1 daca inserarea a reusit
- 0 daca elementul exista deja
- 1 daca nu exista spatiu pentru o noua celula de lista

Tabele de dispersie – operații specifice

Inserare

Preconditie: utilizatorul a alocat si completat corect spatiul ocupat de element.

Observatii:

- daca pentru **verificarea identitatii a doua elemente** se utilizeaza o functie speciala, atunci adresa sa trebuie memorata in descriptorul tabelii; de obicei aceasta functie se refera strict la sirul cheie
- noua celula poate fi **inserata** in lista **la inceput, la sfarsit sau pe baza unei relatii de ordine**, in functie de varianta de implementare aleasa; in ultima varianta adresa functiei care verifica relatia de ordine trebuie memorata in descriptorul tabelii.

Tabele de dispersie – operații specifice

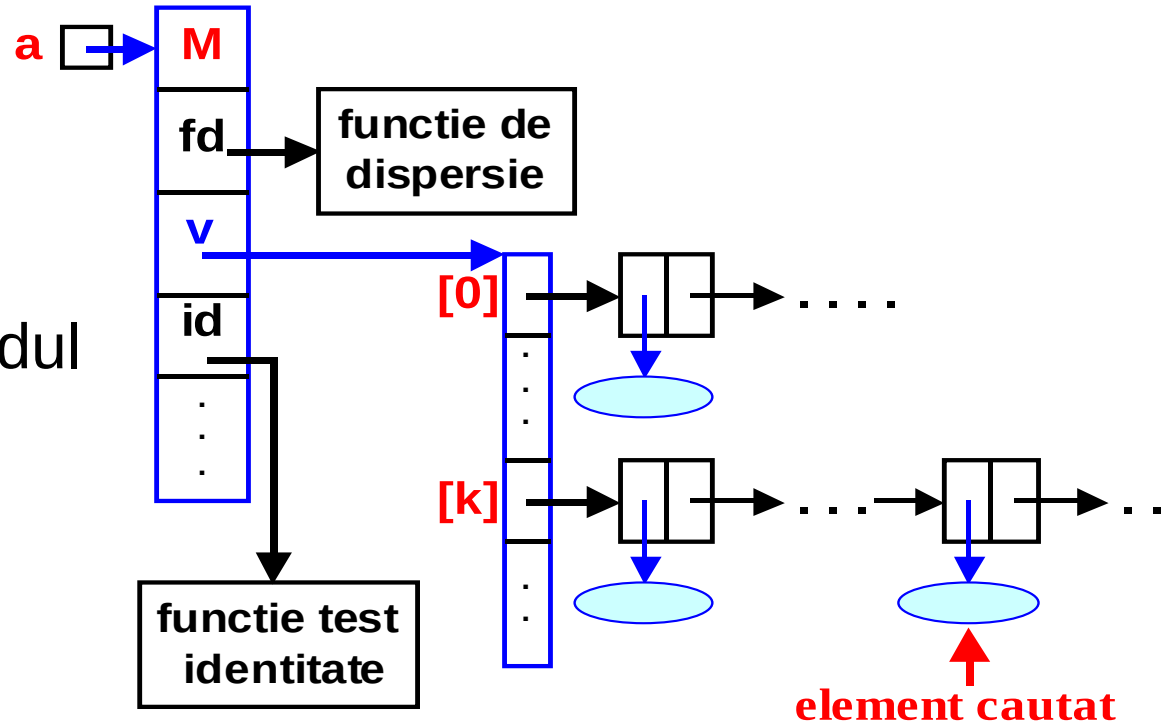
Cutare:

➤ parametri:

- adresa tabela
- (adresa) element

➤ **efect**: calculeaza codul de dispersie **k** si cauta elementul in lista **[k]** (utilizand functia de verificare a identitatii elementelor)

➤ **rezultat**: adresa element / NULL

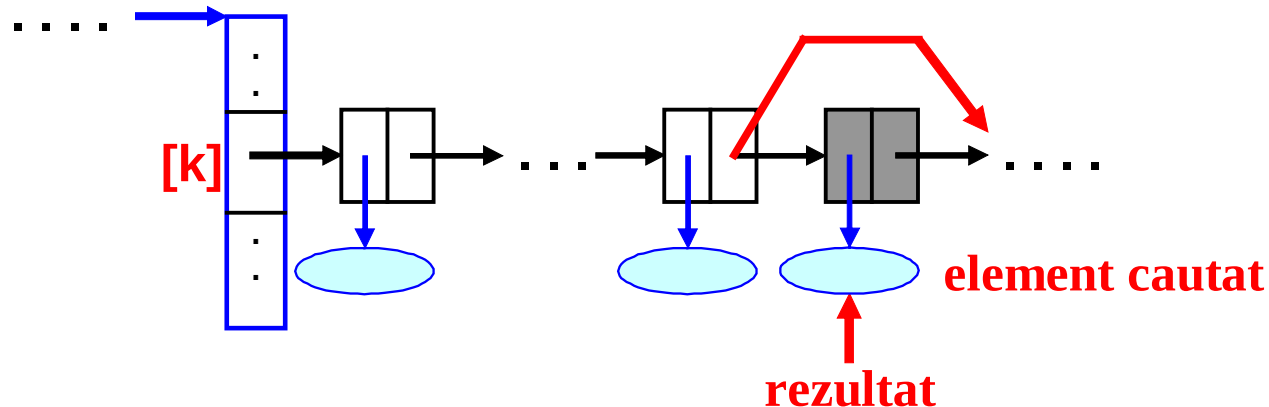


Timpul de căutare mediu este **$O(1)$** iar cel mai nefavorabil (greu de atins) **$O(N)$**

Tabele de dispersie – operații specifice

Extragere/eliminare:

- **parametri**: adresa tablei si (adresa) element
- **efect**: calculeaza codul de dispersie **k** si actualizeaza lista **[k]**, daca elementul este gasit (spatiul ocupat de element nu este afectat, eventuala eliberare a acestuia fiind responsabilitatea utilizatorului)
- **rezultat**: adresa element / NULL



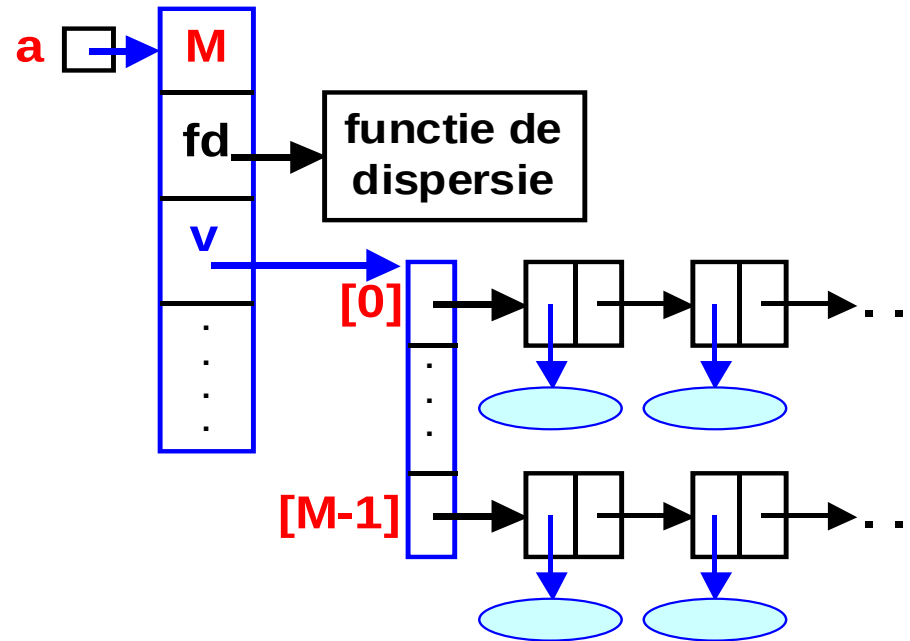
Tabele de dispersie – operații specifice

Stergere tabela:

➤ **parametri:**

- adresa tablei

➤ **efect:** elibereaza mai intai tot spatiul ocupat de liste si apoi spatiul alocat la initializare



4. Funcții de dispersie

Funcția trebuie să fie **deterministă** și **uniformă**

Cea mai comuna metodă de hashing pentru o cheie **k** este

$$h(k) = k \bmod M$$

dacă cheia este un număr întreg

```
typedef long (*HashFunction)(int, long);
```

Dacă cheia este **un șir de caractere de dimensiune mică** și cunoscută, atunci caracterele pot fi văzute ca șiruri de biți iar șirul interpretat ca număr

```
typedef long (*HashFunction)(char*, long);
```

Funcții de dispersie

Altă soluție

Litera de pe poziția i din alfabet este reprezentată de un **cod de 5 biți**

A K E Y
00001 01011 00101 11001

este echivalent cu **44217** în zecimal.

Fie $M = 101$, $44217 \bmod 101 = 80$ ($44137 = 101 * 437$)

Deci cheia AKEY are codul hash 80

(Dar si B A R H are codul hash 80)

Dimensiunea 101 este mica

Funcții de dispersie

Este bine ca dimensiunea tabelii de dispersie sa fie un număr prim.

In alte cazuri o dimensiune multiplu de 2 poate fi avantajoasă

Functia hash modulo M. Daca M este o putere a lui 2 atunci acelasi efect printr-o operatie de mask

Alta posibilitate

Fiecare digit îl reprezentăm ca un număr în baza 32

$$1 * 32^3 + 11 * 32^2 + 5 * 32^1 + 25 * 32^0$$

A K E Y

$$M = 32$$

Funcția de dispersie a fiecărui caracter este codul ultimului caracter – nu este uniformă

Funcții de dispersie

Cazul în care cheia este un șir de caractere de dimensiune mai mare

Este bine ca funcția hash să depindă atât de caractere cât și de poziția lor în șir

$$(\sum_{j=1, \text{strlen}(s)} s(j) * (j-1)) \% M$$

unde $s(j)$ este caracterul j din șir

În esență este o sumă ponderată cu poziția în șir a codului caracterelor din șir

Funcții de dispersie

Folosind reprezentarea caracterelor cu 5 biți și poziția în alfabet

VERYLONGKEY

$$22 * 32^{10} + 5 * 32^9 + 18 * 32^8 + 25 * 32^7 + 12 * 32^6 + 15 * 32^5 + 14 * 32^4 + 7 * 32^3 + 11 * 32^2 + 5 * 32^1 + 25 * 32^0$$

Număr prea mare

Transformăm treptat cheia

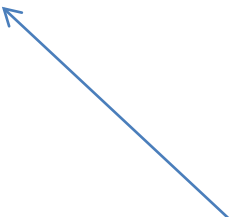
$$((((((((((22 * 32 + 5) * 32 + 18) * 32 + 25) * 32 + 12) * 32 + 15) * 32 + 14) * 32 + 7) * 32 + 11) * 32 + 5) * 32 + 25$$

și aplicăm modulo după fiecare paranteză

Funcții de dispersie

```
unsigned one_hash_function(char *v)
{ int h;
  for (h = 0; *v != '\0' ; v++)
    h = (32 * h + *v) % M;
  return h;
}
```

sau 64

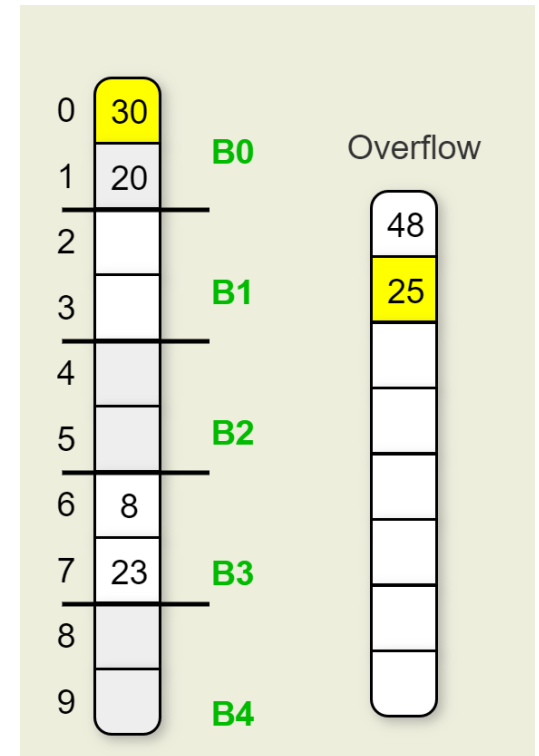
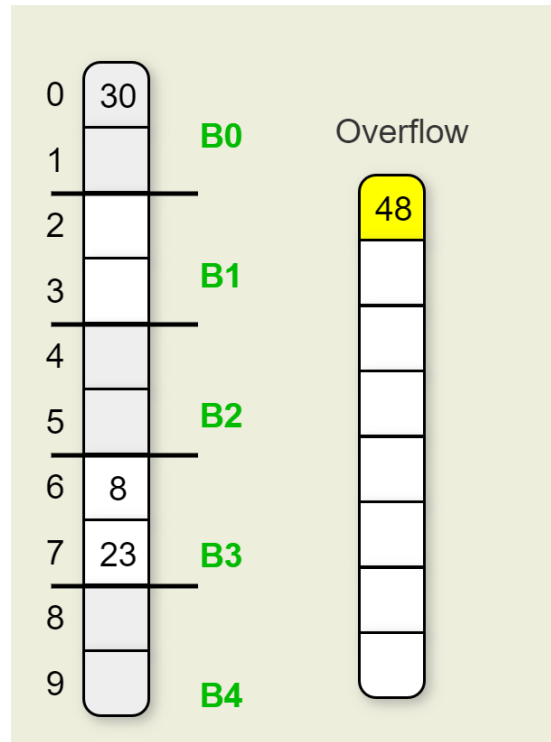
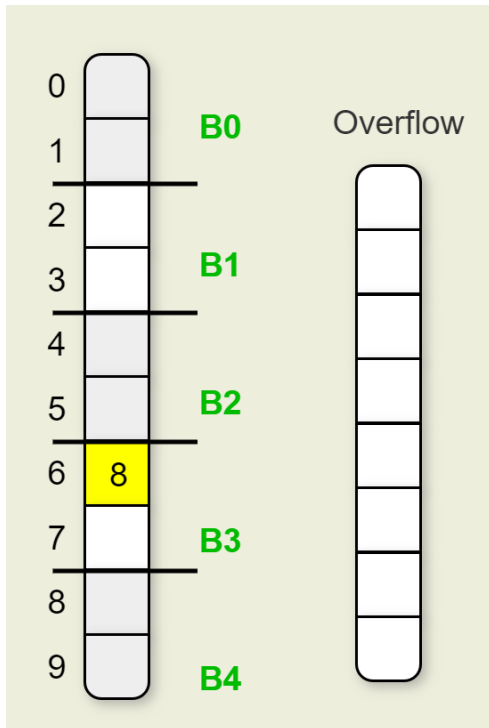


Rezolvarea coliziunilor

- Dispersie deschisa (open hashing) – cea prezentata anterior, cu liste inlantuite
- Dispersie inchisa (closed hashing sau open adressing) – memoreaza inregistrările (sau cheile si pointer la inregistrari) chiar in tabela hash
De ex: Dispersie deschisa cu grupuri (bucket hashing)
- Grupuri de M/B celule in tabela hash, B - numar de grupuri + un grup de overflow (overflow bucket)

Bucket hashing

$M=10$, $B=5$ $h(k) = k \bmod B$



5. Aplicații

Foarte multe clase de aplicații ale TD

- Compilatoare – tabele de simboluri
- Determinarea rapidă a elementelor cu chei egale în colecții mari de date
- Determinarea rapidă a grupurilor de elemente cu chei similare
- Algoritmul de căutare Rabin-Karp a șirurilor de caractere în șiruri lungi de caractere (string pattern match)
- Algoritmul de compresie a șirurilor de caractere LZW
- etc.

Algoritmi de căutare în șiruri

Prima aparitie / toate aparitiile a unui **sir p**
de M caractere (**șabon**) intr-un **sir a de N**
caractere (**text**)

N este foarte mare, M poate fi mare

Cum difera cautarea in sir a unui sablon de
cautarea unei chei

Algoritmi de căutare în șiruri

1. **Algoritmul cautării directe:** Cazul cel mai nefavorabil – timp proportional cu $N \cdot M$; în multe cazuri practice, timpul mediu este proportional cu $N + M$.
2. 1970 - **D.E. Knuth, J.H. Morris** și **V.R. Pratt** au construit un algoritm de cautare rapida
3. 1976 - **R.S. Boyer** și **J.S. Moore** au descoperit un algoritm mai rapid decat (2) în multe aplicatii, deoarece examineaza în multe cazuri numai o parte din caracterele textului.
4. 1980 - **R.M. Karp** și **M.O. Rabin** au observat ca problema căutării în șiruri nu este mult diferita de o problema standard de cautare și au descoperit un algoritm care are virtual timpul proportional cu $M + N$.

Algoritmul căutării directe

```
int cautsablon1(char *p, char *a)
{ int i = 0, j = 0;    /* i este pointerul in text */
  int M = strlen(p);  /* j este pointerul in sablon*/
  int N = strlen(a);

do
{
  while ((i < N) && (j<M) && (a[i]==p[j])) {i++;j++;}
  if (j==M) return i-M;
  i- = j-1;
  j = 0;
} while (i<N);
return i;
}
```

Algoritmul căutării directe

```
int cautsablon2(char *p, char *a)
{
    int i, j;
    int M = strlen(p);
    int N = strlen(a);

    for (i=0, j=0; j<M && i<N; i++, j++)
        while (a[i] != p[j])
        {
            i- = j-1;
            j = 0;
        }
    if (j==M)
        return i-M;
    else
        return i;
}
```

Algoritmul căutării directe

- **Proprietate:** Algoritmul cautării directe necesita, în cazul cel mai nefavorabil, un număr de **maxim $N \cdot M$** comparații de caractere.
- Timpul de rulare pentru cazul cel mai defavorabil este proportional cu $M \cdot (N - M + 1)$, și cum $M \ll N$ obținem valoarea aproximativă $N \cdot M$.
- Cazul cel mai nefavorabil poate apărea, de exemplu, la prelucrarea imaginilor binare.
- În multe aplicații de prelucrare a textelor, bucla interioară se execută rareori și timpul de rulare este proportional cu $N + M$.

Algoritmul Rabin-Karp

Idee

- Considera textul ca fiind o memorie mare si trateaza fiecare secventa de M caractere a textului ca o cheie intr-o tabela de dispersie (hash).
- Trebuie sa calculam functia de dispersie pentru toate secventele posibile de M caractere consecutive din text si sa verificam daca valorile obtinute sunt egale cu functia de dispersie a sablonului.
- Artificiu pt. a nu tine toata tabela de dispersie in memorie.

Calculul funcției hash

- $h(k) = k \bmod q$, unde k este codificarea ca număr a caracterelor din sir, iar q (dimensiunea tabelii) este un număr prim mare.
- q poate fi chiar foarte mare deoarece tabela hash nu se stochează în memorie
- **Funcția de dispersie pentru poziția i din text se calculează pe baza valorii acestei funcții pentru poziția $i-1$**
- Se transformă grupuri de M caractere în numere, prin împachetare ca reprezentare într-o anumită bază
- Aceasta corespunde la a scrie caracterele ca numere în baza d , unde d este numărul de caractere posibile din alfabet.

Calculul funcției hash

- O secvența de M caractere consecutive $a[i] \dots a[i+M-1]$ corespunde:

$$y = a[i] \cdot d^{M-1} + a[i+1] \cdot d^{M-2} + \dots + a[i+M-1]$$

- Presupunem că știm valoarea lui $h(y) = y \bmod q$.
- Deplasarea în text cu o poziție la dreapta corespunde înlocuirii lui y cu

$$y_1 = (y - a[i] \cdot d^{M-1}) \cdot d + a[i+M]$$

- Considerăm $d=32$ (sau orice multiplu de 2 corespunzător)
- Cum facem să nu memorăm valorile funcției hash pentru toate grupurile de M caractere din sir?

Calculul funcției hash pt. y_1 pe baza lui y

Se cunoaste ca:

$$(a + b) \bmod q = (a \bmod q + b \bmod q) \bmod q$$

$$(a * b) \bmod q = (a * (b \bmod q)) \bmod q$$

Se doreste sa se arate ca:

$$(a_0 * x + a_1) \bmod q = ((a_0 \bmod q) * x + a_1) \bmod q$$

Demonstratie:

$$\begin{aligned} (a_0 * x + a_1) \bmod q &= ((a_0 * x) \bmod q + a_1 \bmod q) \bmod q = \\ &= [(x * (a_0 \bmod q)) \bmod q + a_1 \bmod q] \bmod q = ((a_0 \bmod q) * x + a_1) \bmod q \end{aligned}$$

Noul sir impachetat y_1 se obtine din y astfel (slide precedent):

$$y_1 = (y - a[i] * d^{M-1}) * d + a[i+M]$$

Cum se calculeaza $y_1 \bmod q$ pe baza lui y :

$$\begin{aligned} y_1 \bmod q &= ((y - a[i] * d^{M-1}) * d + a[i+M]) \bmod q \\ &= ((y - a[i] * d^{M-1}) \bmod q * d + a[i+M]) \bmod q \end{aligned}$$

Algoritmul Rabin-Karp

```
int caut_RK(char *p, char *a)
{
    int i;      long int dM=1, h1=0, h2=0;
    int M=strlen(p), N=strlen(a);
    for (i=1; i<M; i++) dM = (d*dM) % q;
                        // calculeaza  $d^{(M-1)} \bmod q$  in dM
    for (i=0; i<M; i++)
    {
        h1 = (h1*d + index(p[i]))%q;
                        // functia hash pt sablon
        h2 = (h2*d + index(a[i]))%q;
                        // functia hash pt text
    }
    for (i=0; h1!=h2; i++)
    {
        h2 = (h2 + d*q - index(a[i])*dM)%q;
                // d*q se adauga pt a mentine expresia >0
        h2 = (h2 * d + index(a[i+M]))%q;
        if (i > (N-M)) return N;
    }
    return i;
}
```

$$y_{i+1} \bmod q = (y_i * d + a[i+1]) \bmod q$$

$$y1 \bmod q = ((y - a[i] * d^{M-1}) \bmod q * d + a[i+M]) \bmod q$$

Caracteristici RK

- Algoritmul Rabin-Karp are destul de probabil o complexitate timp liniara.
- Algoritmul are timpul proportional cu $N+M$, dar el gaseste doua siruri cu valori de dispersie egale.
- In aceste conditii, trebuie sa comparam sablonul cu cele M caractere din sir, pentru cazul coliziunilor.
- Deci, din punct de vedere teoretic, acest algoritm ar putea sa fie tot **$O(NM)$** , in cazul cel mai defavorabil, in care toate sabloanele gasite ar fi coliziuni $\rightarrow$ putin probabil, deoarece q este mare; in practica, algoritmul foloseste **$N+M$** pasi.

Compresie Lempel-Ziv-Welch

- Este un algoritm de compresie a șirurilor de caractere foarte folosit
- Metoda folosește un dicționar prin care asociază unor șiruri de caractere de lungimi diferite coduri numerice întregi și înlocuiește secvențe de caractere din fișierul inițial prin aceste numere.
- Acest dicționar este cercetat la fiecare nou caracter extras din fișierul inițial și este extins de fiecare dată când se găsește o secvență de caractere care nu exista anterior în dicționar.
- Dimensiunea uzuală a dicționarului este 4096, dintre care primele 256 de poziții conțin toate caracterele individuale ce pot apărea în fișierele de comprimat.

Compresie Lempel-Ziv-Welch

- Șirul inițial (de comprimat) este analizat și codificat într-o singură trecere, fără revenire.
- La stânga poziției curente sunt subșiruri deja codificate, iar la dreapta cursorului se caută cea mai lungă secvență care există deja în dicționar.
- Odată găsită această secvență, ea este înlocuită prin codul asociat deja și se adaugă la dicționar o secvență cu un caracter mai lungă.
- Dicționarul folosit este implementat ca o **tabelă hash**

Algoritmul de compresie LZW

1. inițializează dicționarul cu toate caracterele din șir și codurile asociate
2. $P = \{ \}$
3. C este caracterul curent din șir
4. **cat timp** C nu este ultimul caracter **repeta**
 daca $P+C$ este in dicționar
 atunci $P = P+C$
 altfel /* $P+C$ nu este in dicționar */
 determina codul lui P din dicționar și
 depune-l in sirul de iesire
 adauga $P+C$ in dicționar
 $P = C$
 C trece la caracterul urmator
5. depune in sirul de iesire codul asociat lui P
6. sirul de iesire este codificarea dorita
sfarsit

Intrare: w a b b a w a b b a

1 a

2 b

3 w

$P = \{\}$ $C = w$ $P+C = w$

$P = w$ $C = a$ $P+C = wa$

$P = a$ $C = b$ $P+C = ab$

$P = b$ $C = b$ $P+C = bb$

$P = b$ $C = a$ $P+C = ba$

$P = a$ $C = w$ $P+C = aw$

$P = w$ $C = a$ $P+C = wa$

$P = wa$ $C = b$ $P+C = wab$

$P = b$ $C = b$ $P+C = bb$

$P = bb$ $C = a$ $P+C = bba$

$P = a$

Ieșire: 3 1 2 2 1 4 6 1

Dictionary

adaug 4 wa output 3

adaug 5 ab output 1

adaug 6 bb output 2

adaug 7 ba output 2

adaug 8 aw output 1

adaug 9 wab output 4

adaug 10 bba output 6

output 1

Algoritmul de decompresie LZW

1. inițializează dicționarul cu toate caracterele din șir și codurile asociate
2. cw – primul cod din sirul de intrare
3. Scrie la iesire sir(cw)
4. pw=cw
5. cw – urmatorul cod
6. **daca** cw este in dicționar **atunci**
 - scrie la iesire sir(cw)
 - P = sir(pw)
 - C = primul caracter din sir(cw)
 - adauga P+C in dicționar**altfel**
 - P = sir(pw)
 - C = primul caracter din sir(pw)
 - scrie la iesire P+C si adauga-l in dicționar
7. **daca** mai sunt coduri in sirul de intrare **atunci** repeta de la 4 **altfel** stop

Intrare: 3 1 2 2 1 4 6 1

1 a

2 b

3 w

4 wa

5 ab

$cw=3$ $\text{sir}(cw) = \mathbf{w}$

$pw=3$

$cw=1$ $\text{sir}(cw) = \mathbf{a}$

$P=\text{sir}(pw)=w$

$C=\text{sir}(cw)=a$

$P+C = wa$

$pw=1$

$cw=2$ $\text{sir}(cw) = \mathbf{b}$

$P=\text{sir}(pw)=a$

$C=\text{sir}(cw)=b$

$P+C=ab$

leşire: **w a b ...**

Algoritmul Knuth-Morris-Pratt

Descoperit independent de:

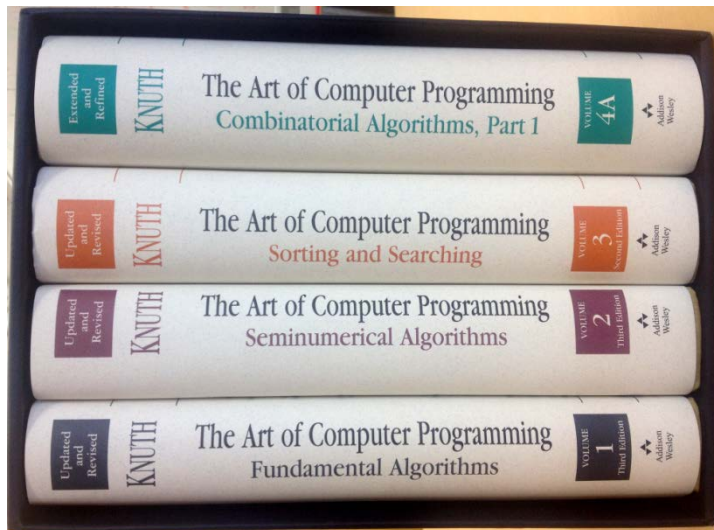
- *Donald Ervin Knuth*
- *James Hiram Morris*
- *Vaughan Pratt*

Knuth-Morris-Pratt

Donald Ervin Knuth – profesor emerit la
Stanford University

PhD CalTech

Autor al *The Art of Computer Programming*



Knuth-Morris-Pratt

James Hiram Morris – profesor la Carnegie
Mellon University
Absolvent CMU și MIT



Knuth-Morris-Pratt

Vaughan Pratt – profesor emerit la Stanford University

Absolvent Sydney University



Algoritmul Knuth-Morris-Pratt

Metoda

- reprezentari binare pentru a explica metoda
- startul fals al identificarii consta din caractere deja cunoscute
- a – sir, i – index in sir, a de lungime N
- p – sablon, j – index in sablon, p de lungime M
- **Consideram intai un caz particular:** primul caracter din sablon nu mai apare in sablon

	i	
sir:	100000 <u>1</u> 000000	a
sablon:	100000 <u>0</u>	p
	j	

in loc de $i = i - j + 1$ se considera $i++$ si $j = 0$

Caz general

sir: 1010100111 a

sablon: 10100111 p

$j=4$

- i trebuie decrementat. **Cu cat?**
- Se poate cunoaste aceasta valoare – depinde de caracterele din sablon
- Daca **nepotrivirea** intre sir si sablon apare pe pozitia j in sablon, **si**
- Exista un $e \in [0, j-1]$ pentru care

$$p[0] \dots p[e-1] = p[j-e] \dots p[j-1]$$

atunci o posibila identificare intre sir si sablon poate apare incepand cu caracterul din **sir** $a[i-e]$ si caracterul din **sablon** $p[j-e]$.

Pt sablon exemplu $j = 4, e=2$

$$p[0] p[2-1] = p[4-2] p[4-1], \text{ adica } p[0] p[1] = p[2] p[3]$$

Caz general

Trebuie cautat **e maxim** cu aceasta proprietate.

- Daca **nu exista e** cu ac. proprietatea atunci cautarea se reia de la **pozitia i si j=0**.
- Daca **exista** un astfel de **e** atunci
$$i = i - e, j = 0 .$$
- Se calculeaza valorile unui vector **next[j]** pt $\forall p_j \in p$ (cu M elemente)
= cu cat trebuie decrementat i la aparitia unei nepotriri pe pozitia j
- **next[j] = e_{max}** – pentru $j > 0$, $e_{\max}$ este numarul maxim de caractere **e < j** de la inceputul sablonului care identifica cu ultimele **e** caractere din primele **j** caractere din sablon

Vectorul next

$i=4$

1010100111

10100111

$j = 4$

$\text{next}[4] = 2 = e_{\max}$

Reluare cautare $i = i - \text{next}[j]$, $j = 0$

Cum se calculeaza $\text{next}[j]$?

- Se gliseaza o copie a primelor j caractere din sablon peste sablon *de la stanga la dreapta*, incapand cu primul caracter din copie suprapus cu al doilea caracter din sablon
- Se opreste glisarea daca:
 - (a) toate caracterele suprapuse (pana la $j-1$) coincid
 - sau
 - (b) primul caracter al copiei ajunge pe pozitia j .

Vectorul next

- Caracterele identice definesc urmatorul loc posibil in care sablonul poate identifica cu sirul, dupa ce se gaseste o nepotrivire pe pozitia j (caracterul $p[j]$).
- Distanța cu care trebuie sa ne intoarcem in sir este exact **next[j]**, adica numarul de caractere care coincid.
- Este convenabil **next[0] = -1**.

Cum facem sa nu ne intoarcem in sir?

- $a[i] \neq p[j] \Rightarrow$ se reia de la $i - \text{next}[j]$ si $j = 0$
- Dar **primele next[j]** caractere incepand de la pozitia $i - \text{next}[j]$ din sir = **primele next[j]** caractere din sablon
 $\Rightarrow$ **i nemodificat** si **j = next[j]**

Exemplu

$i = 4$

ar trebui $i = 2$ si $j = 0$

sir: 1010100111 a

sablon: 10100111 p

$j=4$

$next[4] = 2$

acelasi efect daca i nemodificat si $j = 2$

sir: 1010100111 a

sablon: 10100111 p

Algoritmul Knuth-Morris-Pratt

```
int cautKMP(char *p, char *a)
{
    int i, j;
    int M=strlen(p);
    int N=strlen(a);

    initnext(p);
    for (i=0, j=0; j<M && i<N; i++, j++)
        while ((j>=0) && (a[i] != p[j]))
            j = next[j];
    if (j==M)
        return i-M; /* am gasit potrivire */
    else
        return i;
}
```

Obs: Pt. $j = 0$ si $a[i] \neq p[0]$ este necesar ca $i+$, $j = 0$ deci $next[0] = -1$

Calculul vectorului next

```
void initnext(char *p)
{ int i,j, M=strlen(p);

  next[0] = -1;
  for (i=0, j=-1; i<M; i++, j++, next[i]=j)
    while ((j>=0) && (p[i] != p[j]))
      j=next[j];
}
```

Obs:

- Dupa incrementarile i++ si j++ avem $p[i-j] \dots p[i-1] = p[0] \dots p[j-1]$
- Identificarea efectueaza maxim $M+N$ comparatii de caractere

Caracteristici KMP

- Algoritmul **KMP** presupune precompilarea sablonului, ceea ce este justificat dacă $M \ll N$, adică textul este considerabil mai lung decât sablonul;
- Algoritmul **KMP** nu decrementează indexul din sir, proprietate folosită mai ales pentru căutările pe suport extern;
- Algoritmul **KMP** da rezultate bune dacă o nepotrivire a apărut după o identificare parțială de o anumită lungime. În cazul în care aceste situații sunt o excepție, algoritmul nu aduce îmbunătățiri semnificative;
- Algoritmul **KMP** îmbunătățește timpul cazului cel mai nefavorabil ($N+M$), dar nu și timpul mediu.